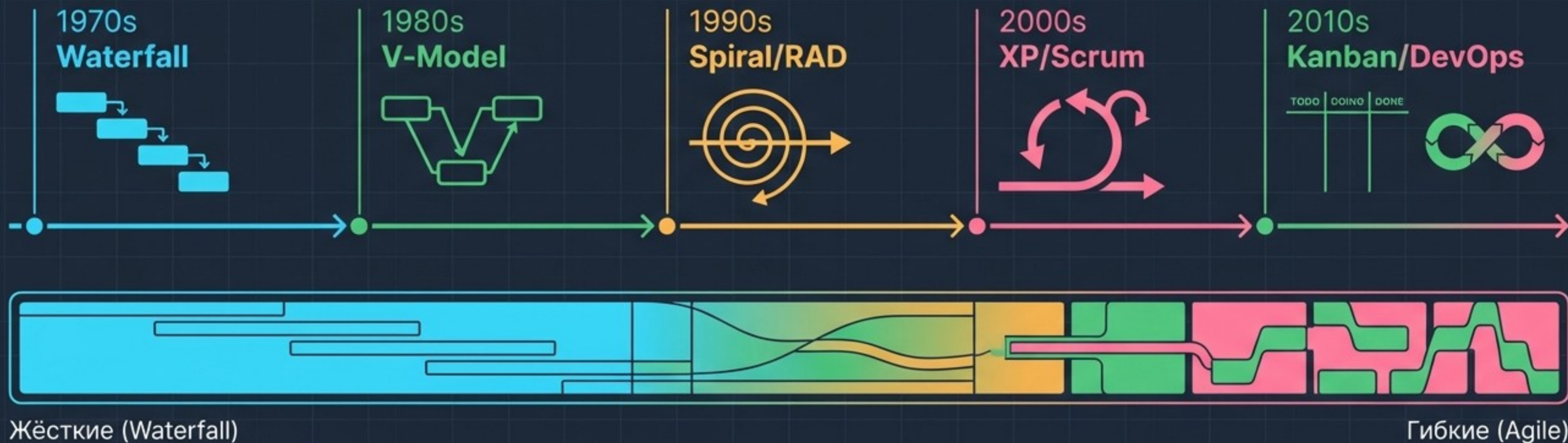


# Методологии разработки ПО

От SDLC до Agile: выбираем подход для QA

Шпаргалка для QA-специалистов. Понимание методологии — ключ к эффективному тестированию. Каждая методология предъявляет уникальные требования к QA-процессу.



**KEY INSIGHT:** Нет универсальной методологии. Выбор зависит от контекста проекта и стабильности требований.

# SDLC (Software Development Life Cycle) — Фундамент

Структурированный процесс от концепции до вывода из эксплуатации.



## Интенсивность участия QA



**QA-INSIGHT:** QA участвует с первого дня!  
Раннее обнаружение дефектов = дешевое исправление.

# Waterfall — Каскадная модель

Классическая последовательность. Водопад течет только вниз.



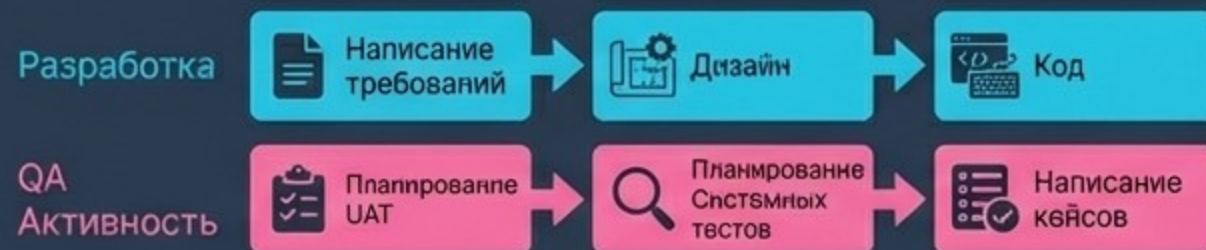
## Waterfall vs Agile

| Требования                                                                                                         | Продукт                                                                                                 | Риск |
|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------|
|  Фиксированные                  |  Эволюционирующие    |      |
|  Виден в конце                  |  Виден каждый спринт |      |
|  Высокий (позднее обнаружение) |  Средний            |      |

Идеально для медицины и авиации.  
Губительно для стартапов.

# V-Model — Верификация и Валидация

Тестирование зеркально разработке.



В Waterfall тестируют ПОСЛЕ. В V-Model тесты ПЛАНИРУЮТСЯ во время разработки.

# Spiral Model — Фокус на рисках

Снижение неопределенности с каждым витком.



Уровень риска vs Время

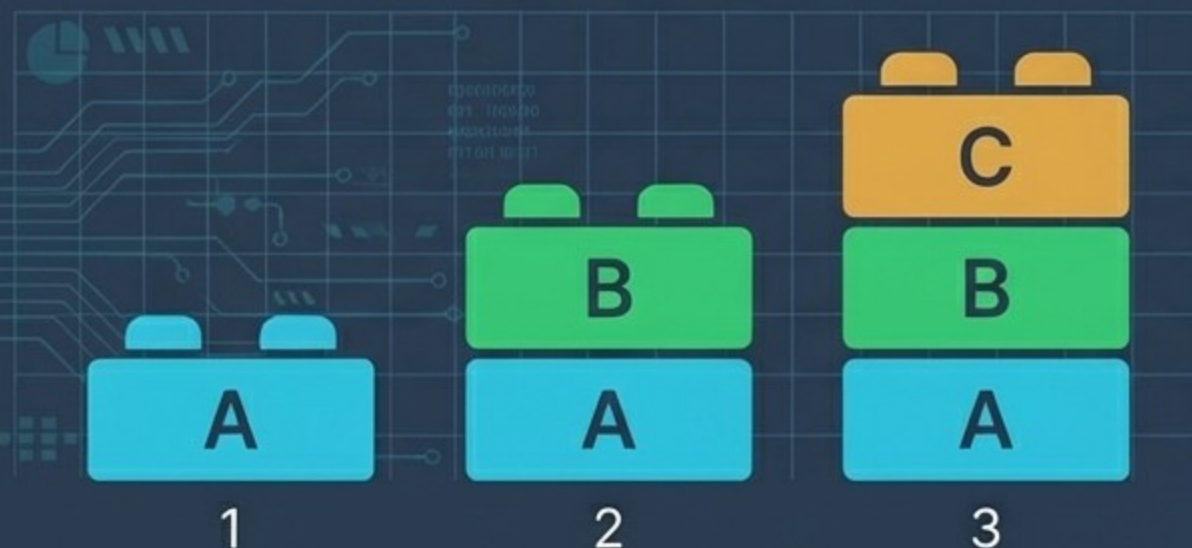


**Контекст:** Используется в NASA, военных проектах и сложных системах безопасности.

# Incremental (По частям) vs Iterative (Улучшения)

Разные стратегии эволюции продукта.

## Incremental Model

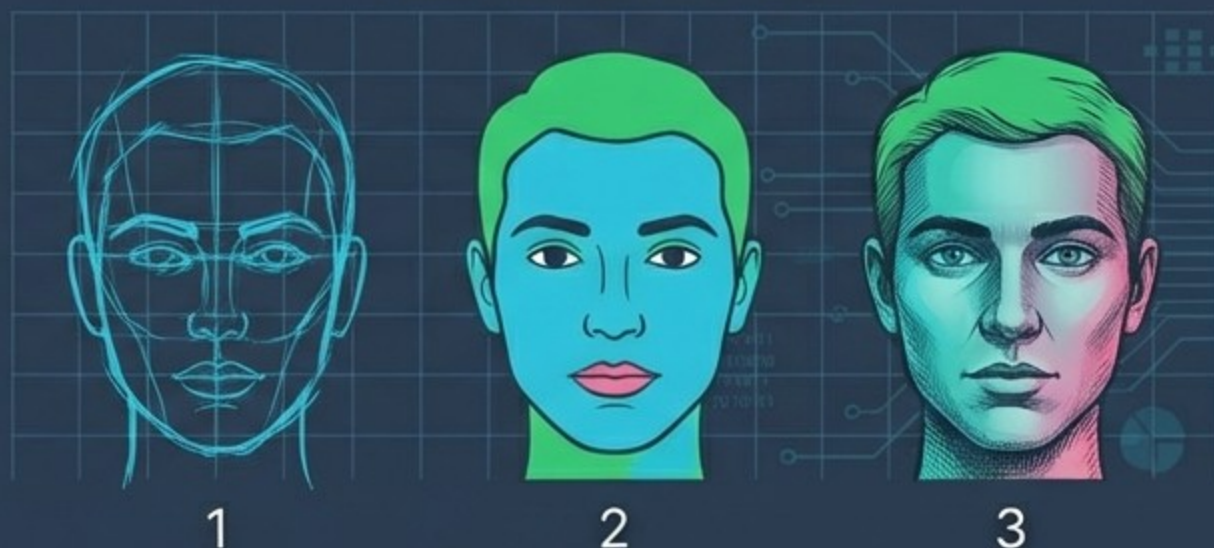


Добавление нового функционала.

JetBrains Mono

**JetBrains Mono:** Растущий объем регрессии.

## Iterative Model



Улучшение существующего.

JetBrains Mono

**JetBrains Mono** – Глубокое тестирование изменений.

# RAD — Rapid Application Development

Быстрая разработка (60-90 дней) и прототипы.



JetBrains Mono

**Target:** Идеально для внутренних инструментов и MVP. Не подходит для сложной архитектуры.

# Сводная таблица методологий

| Методология | Гибкость  | Документация | Управление рисками | Команда  |
|-------------|-----------|--------------|--------------------|----------|
| Waterfall   | ● Низкая  | ● Высокая    | ● Низкое           | Любая    |
| V-Model     | ● Низкая  | ● Высокая    | ● Среднее          | Любая    |
| Spiral      | ● Средняя | ● Высокая    | ● Высокое          | Большая  |
| RAD         | ● Высокая | ● Низкая     | ● Среднее          | Малая    |
| Agile       | ● Высокая | ● Низкая     | ● Среднее          | 3-9 чел. |



JetBrains Mono

**Чёткие требования?** -> Waterfall.

**Нужен MVP?** -> RAD.

# Agile Manifesto — Философия гибкости

4 Ценности, изменившие индустрию (2001).

## Люди и взаимодействие

Люди и взаимодействие  
важнее  
процессов и  
инструментов



## Работающий продукт

Работающий продукт  
важнее  
исчерпывающей  
документации



## Сотрудничество с заказчиком

Сотрудничество  
с заказчиком  
важнее  
контрактов



## Готовность к изменениям

Готовность  
к изменениям  
важнее  
следования  
плану



JetBrains Mono

**Traditional QA:**  
Gatekeeper  
(Вратарь)



Смена парадигмы

JetBrains Mono

**Agile QA:**  
Partner  
(Партнер)

# 12 Принципов Agile на практике

## Customer Focus



Ранняя  
доставка



Приветствие  
изменений



Частые  
релизы

## Team Focus



Бизнес +  
Разработчики



Мотивирован-  
ные люди



Лицом к лицу

## Technical Focus



Работающий  
продукт



Устойчивый  
темп



Техническое  
совершенство

## Improvement Focus



Простота



Самооргани-  
зация



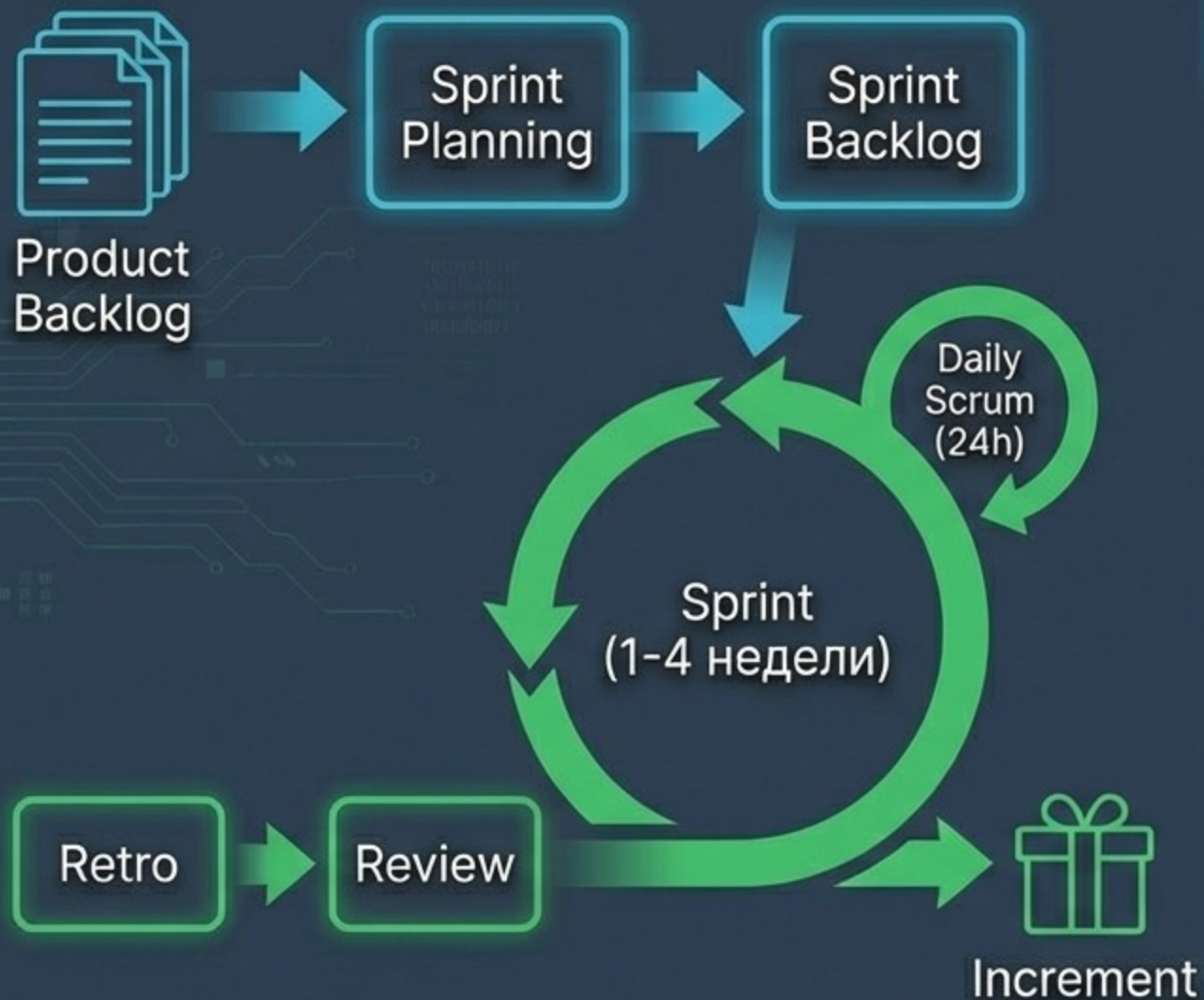
Ретроспектива

JetBrains Nono

**QA Insight:** QA не ждет конца проекта. QA проверяет продукт каждый день.

# Scrum — Спринты и Роли

JetBrains Hono



## 3 Роли

Product Owner  
Scrum Master  
Team

## 3 Артефакта

Product Backlog  
Sprint Backlog  
Increment

## 4 События

Planning  
Daily  
Review  
Retro

Effort Remaining



Time

JetBrains Hono

QA пишет критерии приемки (Acceptance Criteria) **до** старта разработки.

# Kanban — Визуализация и Поток

Stop starting, start finishing.



JetBrains Mono

## Metrics

→ **Lead Time**: Время от идеи до релиза.

⚙️ **Cycle Time**: Время в работе.

JetBrains Mono

## Use Case

Лучше всего для поддержки и Ops.

# Битва титанов: Scrum или Kanban?



**Scrum**



**Ритм:** Фиксированные спринты



**Роли:** Чёткие (PO, SM, Team)



**Изменения:** Только между спринтами



**Метрики:** Velocity



**Kanban**



**Ритм:** Непрерывный поток



**Роли:** Гибкие



**Изменения:** В любой момент (если позволяет WIP)



**Метрики:** Cycle Time



Новый продукт?

Yes



**Scrum**

Поддержка/Ops?

Yes



**Kanban**

# Эволюция тестирования и цена ошибки

## QA Интенсивность

Waterfall

Agile

Crunch

## Цена исправления бага

Экспоненциальный рост

\$1

\$10

\$100

\$1,000

\$10,000

Requirements

Design

Dev

Testing

Production

**Сдвиг влево (Shift Left):** от поиска багов к их предотвращению.

# Итоги — Как выбрать методологию?

|                                                                                                                       |                                                                                      |                      |
|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------------------|
|  Регулируемая отрасль (Med/Avia)?    |   | <b>V-Model</b>       |
|  Высокая неопределенность / Стартап? |   | <b>Agile (Scrum)</b> |
|  Поддержка / Maintenance?            |   | <b>Kanban</b>        |
|  Огромный бюджет / Высокий риск?     |   | <b>Spiral</b>        |
|  Нужен MVP за 2 месяца?             |  | <b>RAD</b>           |

**Методология — это инструмент, а не религия. Адаптируйте её под команду.**